

MedIA — Reporte Técnico de Modelado de Base de Datos	Versión	2.0 — Modelo Final
	Plataforma	Azure Database for PostgreSQL
	Total de entidades	40 entidades
	Normativa	NOM-004-SSA3-2012 / NOM-024-SSA3-2012
	Ciclo escolar	8° Semestre IDTS — UNACH, Enero–Junio 2026

Reporte Técnico de Modelado de Base de Datos

Sistema MedIA — ECE Chiapas

Distrito de Salud I – Tuxtla Gutiérrez, Chiapas

Introducción

El presente documento técnico detalla el diseño arquitectónico definitivo de la base de datos para el sistema de Expediente Clínico Electrónico (ECE) denominado MedIA. Este modelo ha sido diseñado para operar sobre Azure Database for PostgreSQL, optimizado para el contexto geográfico y sociocultural de Chiapas, en estricto cumplimiento con las normas NOM-004-SSA3-2012 (Del expediente clínico) y NOM-024-SSA3-2012 (Sistemas de información de registro electrónico para la salud).

La base de datos emplea un enfoque relacional de alta integridad, aprovechando tipos de datos nativos de PostgreSQL como UUID, TIMESTAMPTZ, ENUM, JSONB e INET. El modelo ha evolucionado de 27 entidades originales a 40 entidades, incorporando: (1) control de acceso granular por especialidad y establecimiento, (2) trazabilidad completa de cambios en datos clínicos sensibles, (3) gestión del ciclo de vida de incidentes de seguridad, (4) soporte estructurado para el Sistema de Referencia y Contrarreferencia normado, y (5) catálogos geográficos basados en el INEGI para el domicilio estructurado del paciente.

Sección 1 — Entidades del Modelo (40 entidades)

GRUPO 1 — Identidad Base (1 entidad)

1. personas

Propósito: Entidad base que unifica la identidad de cualquier individuo en el sistema, ya sea paciente o usuario del personal de salud. Elimina duplicidad entre tablas y garantiza coherencia identitaria. El domicilio se almacena de forma estructurada conforme al catálogo INEGI referenciado por la NOM-024-SSA3-2012.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_persona	UUID	PK, DEFAULT gen_random_uuid()	Identificador único global de la persona.
curp	VARCHAR(18)	UNIQUE, NULL	CURP opcional. Contexto Chiapas: poblaciones indígenas pueden carecer de ella al momento de la consulta.
nombre	VARCHAR(100)	NOT NULL	Nombre(s) en mayúsculas según NOM-004 5.10.
primer_apellido	VARCHAR(100)	NOT NULL	Primer apellido desagregado.
segundo_apellido	VARCHAR(100)	NULL	Segundo apellido. Puede ser nulo en comunidades indígenas con un solo apellido.
fecha_nacimiento	DATE	NOT NULL	Fecha de nacimiento requerida para cálculo de edad en el acto médico.
sexo	ENUM	'M','F','INDETERMINAD O', NOT NULL	Sexo biológico. Requerido por NOM-004 para antecedentes ginecoobstétricos.
calle	VARCHAR(150)	NULL	Calle. NULL permitido: comunidades rurales sin nomenclatura.
numero_exterior	VARCHAR(10)	NULL	Número exterior de domicilio.
numero_interior	VARCHAR(10)	NULL	Número interior / depto.
colonia	VARCHAR(100)	NULL	Colonia o barrio.
codigo_postal	CHAR(5)	NULL	Código postal SEPOMEX.
clave_localidad	VARCHAR(10)	FK (cat_localidades), NOT NULL	Localidad INEGI. Permite reportes epidemiológicos por municipio al Distrito.

<code>referencia_geografica</code>	TEXT	NULL	Referencia libre: 'Frente a la cancha', 'Ejido X'. Para comunidades sin dirección formal.
<code>id_lengua_materna</code>	SMALLINT	FK (cat_lenguas_indigenas), NOT NULL	Lengua materna. Activa alerta visual de barrera lingüística en el módulo de encuentros.
<code>telefono_contacto</code>	VARCHAR(20)	NULL	Teléfono de contacto o de familiar.
<code>creado_en</code>	TIMESTAMPZ	DEFAULT NOW()	Marca de tiempo de creación del registro.

GRUPO 2 — Catálogos Geográficos INEGI (3 entidades)

2. cat_estados

Propósito: Catálogo de estados de la República Mexicana basado en la clave INEGI. Se siembra una única vez. Referenciado por la NOM-024-SSA3-2012 para estandarización del domicilio.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>clave_estado</code>	CHAR(2)	PK	Clave INEGI del estado. Ej: '07' = Chiapas.
<code>nombre_estado</code>	VARCHAR(100)	NOT NULL, UNIQUE	Nombre oficial del estado.

3. cat_municipios

Propósito: 124 municipios del estado de Chiapas según catálogo INEGI. Permite reportes epidemiológicos y segmentación geográfica para la coordinación del Distrito.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>clave_municipio</code>	CHAR(3)	PK	Clave INEGI del municipio.
<code>clave_estado</code>	CHAR(2)	FK (cat_estados), NOT NULL	Estado al que pertenece el municipio.
<code>nombre_municipio</code>	VARCHAR(100)	NOT NULL	Nombre oficial del municipio.

4. cat_localidades

Propósito: Localidades urbanas y rurales de Chiapas según catálogo INEGI. Clave para identificar comunidades indígenas sin nomenclatura vial formal.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
clave_localidad	VARCHAR(10)	PK	Clave INEGI de la localidad.
clave_municipio	CHAR(3)	FK (cat_municipios), NOT NULL	Municipio al que pertenece.
nombre_localidad	VARCHAR(150)	NOT NULL	Nombre oficial de la localidad o ejido.
tipo	ENUM	'URBANA','RURAL', NOT NULL	Clasificación para lógica de conectividad y distribución CDN.

GRUPO 3 — Catálogos Clínicos y de Sistema (6 entidades)

5. jurisdicciones_sanitarias

Propósito: Clasificación administrativa regional del estado de Chiapas. El Distrito de Salud I corresponde a Tuxtla Gutiérrez.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
id_jurisdiccion	SMALLINT	PK, NOT NULL	Número de jurisdicción (1 a 10).
nombre_jurisdiccion	VARCHAR(100)	NOT NULL, UNIQUE	Nombre oficial de la jurisdicción.
municipio_sede	VARCHAR(100)	NOT NULL	Municipio sede de la jefatura sanitaria.

6. establecimientos

Propósito: Registro oficial de unidades médicas del Distrito I, identificadas por su CLUES (Clave Única de Establecimientos de Salud). Entidad central de la red asistencial.			
Campo	Tipo PostgreSQL	Restricciones	Descripción

<code>id_establecimiento</code>	UUID	PK, DEFAULT gen_random_uuid()	Identificador interno del establecimiento.
<code>clues</code>	VARCHAR(15)	NOT NULL, UNIQUE	Clave Única de Establecimientos de Salud (CLUES).
<code>nombre_comun</code>	VARCHAR(255)	NOT NULL	Nombre descriptivo del establecimiento.
<code>id_jurisdiccion</code>	SMALLINT	FK (jurisdicciones_sanitarias), NOT NULL	Jurisdicción sanitaria a la que pertenece.
<code>nivel_atencion</code>	ENUM	'PRIMARIO','SECUNDARIO', NOT NULL	Nivel asistencial. Determina lógica de acceso cross-establecimiento.

7. cat_cie10

Propósito: Catálogo maestro de diagnósticos CIE-10 en versión oficial vigente publicada por la DGIS. Garantiza interoperabilidad semántica conforme a NOM-024-SSA3-2012.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_cie10</code>	INTEGER	PK	ID secuencial interno.
<code>codigo_cie</code>	VARCHAR(10)	NOT NULL, UNIQUE	Código alfanumérico CIE-10. Ej: E11.9.
<code>descripcion</code>	TEXT	NOT NULL	Narrativa clínica estandarizada del diagnóstico.

8. cat_medicamentos

Propósito: Vademécum institucional vinculado al Cuadro Básico de Medicamentos de la Secretaría de Salud. ON DELETE RESTRICT evita eliminación de fármacos ya prescritos.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_medicamento</code>	UUID	PK, DEFAULT gen_random_uuid()	Identificador único del fármaco.

codigo_medicamento_ssa	VARCHAR(20)	UNIQUE, NOT NULL	Código SSA. Puente para integración con sistemas de farmacia externos.
nombre_generico	VARCHAR(255)	NOT NULL	Denominación genérica oficial.

9. cat_especialidades_medicas

Propósito: Listado normado de especialidades médicas para validación de cédulas profesionales y control de visibilidad de datos clínicos entre establecimientos.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_especialidad	SMALLINT	PK	ID interno.
nombre_especialidad	VARCHAR(150)	NOT NULL, UNIQUE	Nombre según registro profesional oficial.

10. cat_lenguas_indigenas

Propósito: Registro de variantes lingüísticas oficiales según catálogo INALI. Chiapas cuenta con más de 12 grupos etnolingüísticos. Campo crítico para seguridad del paciente.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_lengua	SMALLINT	PK	ID numérico INALI.
variante_especifica	VARCHAR(255)	NOT NULL	Variante dialectal exacta según clasificación INALI.

11. cat_modulos

Propósito: Catálogo de módulos funcionales del sistema. Reemplaza el JSONB de permisos por una estructura relacional administrable desde

la interfaz de configuración.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
id_modulo	SMALLINT	PK	ID interno.
nombre_modulo	VARCHAR(100)	NOT NULL, UNIQUE	Nombre del módulo. Ej: NOTAS_MEDICAS, PACIENTES, AUDITORIA.
descripcion	TEXT	NULL	Descripción funcional del módulo para el panel de administración.

GRUPO 4 — Configuración de Red Asistencial (2 entidades)

12. establecimientos_especialidades

Propósito: Define qué especialidades tiene habilitadas cada unidad médica. Tabla clave para la lógica de negocio de acceso cross-establecimiento: un establecimiento solo puede acceder a datos de la especialidad que tiene registrada como activa.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id	UUID	PK, DEFAULT gen_random_uuid()	ID interno.
id_establecimiento	UUID	FK (establecimientos), NOT NULL	Establecimiento configurado.
id_especialidad	SMALLINT	FK (cat_especialidades_medicas), NOT NULL	Especialidad habilitada.
activa	BOOLEAN	DEFAULT TRUE	Permite deshabilitar sin eliminar. Soft delete implícito.
fecha_alta	DATE	NOT NULL	Fecha en que se habilitó la especialidad en el establecimiento.
desactivado_en	TIMESTAMPTZ	NULL	Timestamp de desactivación. NULL = activo. Soft delete forense.
desactivado_por	UUID	FK (usuarios_sistema), NULL	Administrador que desactivó la especialidad. Trazabilidad forense requerida por Req 6.
motivo_desactivacion	TEXT	NULL	Justificación de la desactivación para auditoría.

GRUPO 5 — Usuarios y Control de Acceso (5 entidades)

13. usuarios_sistema

Propósito: Gestión de identidad y seguridad del personal médico y administrativo. Vinculado a personas para unificar identidad. Contiene controles de seguridad activa: bloqueo por intentos fallidos y 2FA (TOTP).			
Campo	Tipo PostgreSQL	Restricciones	Descripción
id_usuario	UUID	PK, DEFAULT gen_random_uuid()	ID interno del usuario.
id_persona	UUID	FK (personas), NOT NULL, UNIQUE	Vínculo a la identidad base. Un usuario es siempre una persona registrada.
username	VARCHAR(50)	UNIQUE, NOT NULL	Nombre de usuario para login.
password_hash	TEXT	NOT NULL	Hash seguro de contraseña. Algoritmo Argon2id.
cedula_profesional	VARCHAR(20)	NOT NULL	Obligatoria para habilitación de firma electrónica de notas clínicas.
intentos_fallidos	SMALLINT	DEFAULT 0	Contador de intentos fallidos de login. Al llegar a 5 activa bloqueo temporal.
bloqueado_hasta	TIMESTAMPTZ	NULL	Si no es NULL, el sistema rechaza el login hasta que el timestamp sea pasado.
totp_secret	TEXT	NULL	Secreto cifrado para autenticación TOTP (2FA) compatible con Google/Microsoft Authenticator.
totp_habilitado	BOOLEAN	DEFAULT FALSE	Activa la validación del código TOTP en cada inicio de sesión.
activo	BOOLEAN	DEFAULT TRUE	Permite deshabilitar cuentas sin eliminar su historial de auditoría.

14. roles

Propósito: Definición de roles disponibles en el sistema. Reemplaza roles_permisos con estructura relacional. Los permisos específicos se asignan en permisos_rol.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
id_rol	SMALLINT	PK	ID interno del rol.

nombre_rol	VARCHAR (50)	NOT NULL, UNIQUE	Nombre descriptivo. Ej: MEDICO_GENERAL, MEDICO_ESPECIALISTA, ENFERMERIA, ADMINISTRADOR, AUDITOR_SEGURIDAD.
descripcion	TEXT	NULL	Descripción del alcance del rol para el panel de administración.

15. permisos_rol

Propósito: Matriz CRUD de permisos por rol y módulo. Estructura relacional que reemplaza el JSONB anterior, permitiendo administración visual desde la interfaz sin intervención técnica.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_permiso	UUID	PK	ID de la regla de permiso.
id_rol	SMALLINT	FK (roles), NOT NULL	Rol al que aplica el permiso.
id_modulo	SMALLINT	FK (cat_modulos), NOT NULL	Módulo sobre el que aplica.
puede_crear	BOOLEAN	DEFAULT FALSE	Permiso de creación de registros en el módulo.
puede_leer	BOOLEAN	DEFAULT FALSE	Permiso de lectura de registros en el módulo.
puede_actualizar	BOOLEAN	DEFAULT FALSE	Permiso de actualización. En módulos clínicos casi siempre FALSE por inmutabilidad normativa.
puede_eliminar	BOOLEAN	DEFAULT FALSE	Permiso de eliminación lógica. En módulos clínicos siempre FALSE.

16. usuarios_establecimientos

Propósito: Relación N:M que vincula usuarios con unidades médicas y roles. Un mismo médico puede tener diferentes roles en diferentes establecimientos del Distrito.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_usuario	UUID	FK (usuarios_sistema), NOT NULL	Usuario vinculado.

<code>id_establecimiento</code>	UUID	FK (establecimientos), NOT NULL	Unidad médica donde opera.
<code>id_rol</code>	SMALLINT	FK (roles), NOT NULL	Rol que desempeña en ese establecimiento específico.

17. permisos_especialidad

Propósito: Control granular de visibilidad de datos clínicos por especialidad y rol. Determina qué notas puede ver un médico según la especialidad generadora de la nota, complementando el control por establecimiento.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_permiso_esp</code>	UUID	PK	ID de la regla de visibilidad.
<code>id_rol</code>	SMALLINT	FK (roles), NOT NULL	Rol al que aplica la regla.
<code>id_especialidad</code>	SMALLINT	FK (cat_especialidades_medicas), NOT NULL	Especialidad sobre la que aplica la restricción.
<code>puede_leer</code>	BOOLEAN	DEFAULT FALSE	Visibilidad de notas de esta especialidad para el rol.
<code>puede_escribir</code>	BOOLEAN	DEFAULT FALSE	Capacidad de generar notas en esta especialidad para el rol.

GRUPO 6 — Identidad Clínica del Paciente (3 entidades)

18. pacientes

Propósito: Entidad que materializa la relación asistencial de una persona con el sistema de salud. Una persona puede ser paciente en el momento que requiera atención. El número de expediente es el identificador médico-legal del registro clínico.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_paciente</code>	UUID	PK, DEFAULT gen_random_uuid()	ID único global del expediente del paciente.

id_persona	UUID	FK (personas), NOT NULL, UNIQUE	Vínculo a la identidad base. Garantiza que un paciente es una persona registrada.
numero_expediente	VARCHAR(20)	UNIQUE, NOT NULL	Número de expediente generado por el sistema. Identificador médico-legal requerido por NOM-004.
id_establecimiento_alta	UUID	FK (establecimientos), NOT NULL	Establecimiento donde se abrió el expediente originalmente.
fecha_apertura	DATE	NOT NULL	Fecha de apertura del expediente. Requerida por NOM-004.

19. pacientes_tutores_representantes

Propósito: Registro de representantes legales para pacientes menores de edad o con incapacidad legal. El tutor debe ser una persona registrada en el sistema para garantizar coherencia de identidad.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_representante	UUID	PK	ID interno del registro de representación.
id_paciente	UUID	FK (pacientes), NOT NULL	Paciente representado.
id_persona	UUID	FK (personas), NOT NULL	Persona que ejerce la representación legal. Puede ser también paciente del sistema.
parentesco	VARCHAR(50)	NOT NULL	Relación con el paciente: MADRE, PADRE, TUTOR_LEGAL, OTRO.
documento_legal_url	TEXT	NULL	Referencia a Azure Blob Storage con escaneo del documento que acredita la representación.
activo	BOOLEAN	DEFAULT TRUE	Permite registrar cambios de representante sin eliminar el historial.
eliminado_en	TIMESTAMPTZ	NULL	Soft delete forense. NULL = activo. Req 6 Cómputo Forense: borrado lógico obligatorio.
eliminado_por	UUID	FK (usuarios_sistema), NULL	Usuario que realizó la baja lógica del representante.
motivo_baja	TEXT	NULL	Justificación de la baja. Requerido al ejecutar el borrado lógico.

20. alergias

Propósito: Alertas críticas de seguridad del paciente. Cambios en severidad quedan registrados en historial_cambios por normativa. Campo disponible para todos los médicos que atiendan activamente al paciente, sin restricción de especialidad.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_alergia	UUID	PK	ID único.
id_paciente	UUID	FK (pacientes), NOT NULL	Paciente vinculado.
sustancia	VARCHAR(150)	NOT NULL	Sustancia alergénica: fármaco, alimento, material.
severidad	ENUM	'LEVE','MODERADA','CRITICA', NOT NULL	Nivel de riesgo. Cambios auditados por trigger en historial_cambios.
observaciones	TEXT	NULL	Descripción de la reacción observada.
registrado_por	UUID	FK (usuarios_sistema), NOT NULL	Médico que registró la alergia.
fecha_registro	TIMESTAMPTZ	DEFAULT NOW()	Fecha de registro inicial.
eliminado_en	TIMESTAMPTZ	NULL	Soft delete forense. NULL = activo. Un DELETE físico de una alergia viola NOM-004 y Req 6 Forense. La información permanece para trazabilidad médico-legal.
eliminado_por	UUID	FK (usuarios_sistema), NULL	Médico o administrador que realizó la baja lógica.
motivo_baja	TEXT	NULL	Justificación obligatoria de la eliminación lógica. Ej: 'Error de captura', 'Alergia no confirmada'.

GRUPO 7 — Historia Clínica — Antecedentes (5 entidades)

21. antecedentes_heredofamiliares

Propósito: Historial genético de riesgo patológico. La NOM-004 obliga su inclusión en la historia clínica. Auditado por trigger en historial_cambios.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_antecedente</code>	UUID	PK	ID único.
<code>id_paciente</code>	UUID	FK (pacientes), NOT NULL	Paciente vinculado.
<code>familiar</code>	VARCHAR(50)	NOT NULL	Parentesco del familiar: PADRE, MADRE, ABUELO, HERMANO, etc.
<code>descripcion_patologia</code>	TEXT	NOT NULL	Detalle de la patología familiar con relevancia clínica.
<code>registrado_por</code>	UUID	FK (usuarios_sistema), NOT NULL	Médico que registró el antecedente.
<code>fecha_registro</code>	TIMESTAMPTZ	DEFAULT NOW()	Fecha de registro.
<code>eliminado_en</code>	TIMESTAMPTZ	NULL	Soft delete forense. Req 6: el historial genético no puede eliminarse físicamente.
<code>eliminado_por</code>	UUID	FK (usuarios_sistema), NULL	Usuario que realizó la baja lógica.
<code>motivo_baja</code>	TEXT	NULL	Justificación de la eliminación lógica.

22. antecedentes_patologicos

Propósito: Historia clínica personal del paciente. Incluye antecedentes quirúrgicos, traumáticos, transfusionales, alérgicos a medicamentos. Auditado por trigger.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_antecedente</code>	UUID	PK	ID único.
<code>id_paciente</code>	UUID	FK (pacientes), NOT NULL	Paciente vinculado.
<code>tipo_patologia</code>	VARCHAR(100)	NOT NULL	Categoría: QUIRURGICO, TRAUMATICO, TRANSFUSIONAL, HOSPITALIZACION_PREVIA.
<code>descripcion</code>	TEXT	NOT NULL	Detalle de la patología personal.
<code>fecha_aproximada</code>	DATE	NULL	Fecha aproximada del evento patológico.
<code>registrado_por</code>	UUID	FK (usuarios_sistema), NOT NULL	Médico que registró el antecedente.
<code>eliminado_en</code>	TIMESTAMPTZ	NULL	Soft delete forense. Req 6 y NOM-004: historial personal no eliminable físicamente.
<code>eliminado_por</code>	UUID	FK (usuarios_sistema), NULL	Usuario que realizó la baja lógica.
<code>motivo_baja</code>	TEXT	NULL	Justificación de la eliminación lógica.

23. antecedentes_no_patologicos

Propósito:

Determinantes sociales de la salud: vivienda, hábitos, higiene, alimentación, actividad física. Relevantes para el contexto de comunidades rurales de Chiapas.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_antecedente</code>	UUID	PK	ID único.
<code>id_paciente</code>	UUID	FK (pacientes), NOT NULL	Paciente vinculado.
<code>categoria</code>	VARCHAR(100)	NOT NULL	Categoría: VIVIENDA, HABITACION, HIGIENE, ALIMENTACION, TOXICOMANIAS, ACTIVIDAD_FISICA.
<code>descripcion</code>	TEXT	NOT NULL	Descripción de las condiciones de vida.
<code>registrado_por</code>	UUID	FK (usuarios_sistema), NOT NULL	Médico que registró el antecedente.
<code>eliminado_en</code>	TIMESTAMPTZ	NULL	Soft delete forense. Req 6.
<code>eliminado_por</code>	UUID	FK (usuarios_sistema), NULL	Usuario que realizó la baja lógica.
<code>motivo_baja</code>	TEXT	NULL	Justificación de la eliminación lógica.

24. antecedentes_ginecoobstetricos

Propósito: Datos exclusivos de salud reproductiva femenina. CHECK constraint en el backend valida que el sexo del paciente sea 'F'. Auditado por trigger dado que cambios en gestas o FUM son datos diagnósticos críticos.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_antecedente</code>	UUID	PK	ID único.
<code>id_paciente</code>	UUID	FK (pacientes), NOT NULL	Paciente vinculada. Sexo debe ser 'F'.
<code>gestas</code>	SMALLINT	NOT NULL	Número total de embarazos.
<code>partos</code>	SMALLINT	NULL	Número de partos.
<code>cesareas</code>	SMALLINT	NULL	Número de cesáreas.

abortos	SMALLINT	NULL	Número de abortos.
fecha_ultima_menstruacion	DATE	NULL	Fecha de última menstruación (FUM).
metodo_anticonceptivo	VARCHAR(100)	NULL	Método anticonceptivo actual.
registrado_por	UUID	FK (usuarios_sistema), NOT NULL	Médico que registró los antecedentes.
eliminado_en	TIMESTAMPZ	NULL	Soft delete forense. Req 6 y NOM-004. Datos reproductivos no eliminables físicamente.
eliminado_por	UUID	FK (usuarios_sistema), NULL	Usuario que realizó la baja lógica.
motivo_baja	TEXT	NULL	Justificación de la eliminación lógica.

25. inmunizaciones

Propósito: Registro de vacunas aplicadas según el Esquema Nacional de Vacunación. Permite verificar estado vacunal al momento de la consulta.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_vacuna_registro	UUID	PK	ID único del registro de vacunación.
id_paciente	UUID	FK (pacientes), NOT NULL	Paciente vacunado.
nombre_vacuna	VARCHAR(100)	NOT NULL	Denominación oficial. Ej: BCG, Pentavalente, Influenza.
fecha_aplicacion	DATE	NOT NULL	Fecha de aplicación de la dosis.
dosis	VARCHAR(20)	NULL	Número de dosis. Ej: 1ra, 2da, Refuerzo.
registrado_por	UUID	FK (usuarios_sistema), NOT NULL	Médico o enfermero que registró la vacuna.
eliminado_en	TIMESTAMPZ	NULL	Soft delete forense. El historial vacunal no puede eliminarse físicamente. Req 6.
eliminado_por	UUID	FK (usuarios_sistema), NULL	Usuario que realizó la baja lógica.
motivo_baja	TEXT	NULL	Justificación. Ej: 'Error de captura — vacuna aplicada en otra unidad'.

GRUPO 8 — Núcleo del Acto Médico — Consulta Externa (9 entidades)

26. encuentros_clinicos

Propósito: Tabla base que agrupa todas las interacciones clínicas de una consulta externa. Es la unidad central del acto médico: cada consulta genera un encuentro que contiene notas, signos vitales, diagnósticos, prescripciones y solicitudes de estudio.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_encuentro</code>	UUID	PK, DEFAULT gen_random_uuid()	ID único del encuentro clínico.
<code>id_paciente</code>	UUID	FK (pacientes), NOT NULL	Paciente atendido.
<code>id_establecimiento</code>	UUID	FK (establecimientos), NOT NULL	Unidad médica donde se realiza la consulta.
<code>id_medico_responsable</code>	UUID	FK (usuarios_sistema), NOT NULL	Médico responsable del encuentro.
<code>fecha_inicio</code>	TIMESTAMPTZ	NOT NULL	Hora de apertura del encuentro. NOM-004 exige fecha y hora exactas.
<code>fecha_cierre</code>	TIMESTAMPTZ	NULL	Hora de cierre. NULL indica encuentro en curso.
<code>motivo_consulta</code>	TEXT	NOT NULL	Motivo de consulta en palabras del paciente.
<code>tipo_consulta</code>	ENUM	'PRIMERA_VEZ', 'SUBSECUENTE', NOT NULL	Determina si se requiere historia clínica completa o solo nota de evolución.

27. notas_medicas

Propósito: Tabla base de registros clínicos. Implementa el principio de inmutabilidad mediante el flag `esta_firmada`. Una vez firmada, el trigger `tr_notes_protection` bloquea cualquier modificación conforme a NOM-004-SSA3-2012.

Campo	Tipo PostgreSQL	Restricciones	Descripción
-------	-----------------	---------------	-------------

id_nota	UUID	PK, DEFAULT gen_random_uuid()	ID único de la nota.
id_encuentro	UUID	FK (encuentros_clinicos), NOT NULL	Encuentro al que pertenece la nota.
id_usuario_autor	UUID	FK (usuarios_sistema), NOT NULL	Médico que redacta y firma la nota. Requerido por NOM-004.
tipo_nota	ENUM	NOT NULL	Tipo: EVOLUCION, INGRESO, COMPLEMENTARIA, INTERCONSULTA.
esta_firmada	BOOLEAN	DEFAULT FALSE	Al ser TRUE activa el trigger de inmutabilidad. Equivale a la firma electrónica.
fecha_creacion	TIMESTAMPTZ	DEFAULT NOW()	Timestamp de creación real. Preservado aunque la sincronización sea posterior (zonas rurales).

28. notas_soap_detalle

Propósito:

Contenido clínico estructurado según la metodología SOAP propuesta por Lawrence Weed (1968). Relación 1:1 con notas_medicas. La NOM-004-SSA3-2012 obliga la elaboración de notas de evolución en consulta externa.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_nota	UUID	PK, FK (notas_medicas)	Relación 1:1 con la nota base. Comparte PK.
subjetivo	TEXT	NOT NULL	S: Interrogatorio, síntomas referidos por el paciente, motivo de consulta.
objetivo	TEXT	NOT NULL	O: Exploración física, hallazgos objetivos, signos.
analisis	TEXT	NOT NULL	A: Razonamiento clínico, diagnósticos presuntivos y diferenciales.
plan	TEXT	NOT NULL	P: Plan diagnóstico, terapéutico e indicaciones médicas.

29. notas_enmienda

Propósito:

Correcciones legales a notas firmadas

(Addendum). Implementa el concepto normativo: el error permanece visible y la corrección se agrega con nueva firma. Cumple NOM-004 numeral 5.10.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_enmienda</code>	UUID	PK	ID único de la enmienda.
<code>id_nota_original</code>	UUID	FK (notas_medicas), NOT NULL	Nota firmada que se corrige. Solo notas con esta_firmada=TRUE pueden tener enmiendas.
<code>id_medico_corrector</code>	UUID	FK (usuarios_sistema), NOT NULL	Médico que realiza la corrección. Puede ser el autor original u otro.
<code>narrativa_correccion</code>	TEXT	NOT NULL	Texto de la corrección con identificación expresa del error señalado.
<code>fecha_enmienda</code>	TIMESTAMPTZ	DEFAULT NOW()	Timestamp de la enmienda. Preserva el orden cronológico exigido por NOM-004.

30. diagnosticos_encuentro

Propósito: Vinculación de códigos CIE-10 con el encuentro clínico. Garantiza interoperabilidad semántica. Se permite el registro narrativo adicional para no sustituir el juicio médico.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_diagnostico_rel</code>	UUID	PK	ID de la relación diagnóstica.
<code>id_encuentro</code>	UUID	FK (encuentros_clinicos), NOT NULL	Encuentro vinculado.
<code>id_cie10</code>	INTEGER	FK (cat_cie10), NOT NULL	Código diagnóstico CIE-10 estandarizado.
<code>tipo_diagnostico</code>	ENUM	'PRESUNTIVO','DEFINITIVO', NOT NULL	Clasificación diagnóstica requerida por NOM-004. Cambios auditados en historial_cambios.
<code>descripcion_narrativa</code>	TEXT	NULL	Diagnóstico en texto libre del médico. Complementa el código CIE-10 sin sustituirlo.

31. signos_vitales

Propósito:

Almacenamiento paramétrico de constantes físicas del paciente en el encuentro. Capturado generalmente por enfermería al inicio del encuentro. Auditado por trigger en historial_cambios.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_signos	UUID	PK	ID único.
id_encuentro	UUID	FK (encuentros_clinicos), NOT NULL	Encuentro vinculado.
capturado_por	UUID	FK (usuarios_sistema), NOT NULL	Personal que realizó la toma de signos vitales.
peso_kg	DECIMAL(5,2)	NOT NULL	Peso en kilogramos.
talla_cm	DECIMAL(5,2)	NOT NULL	Estatura en centímetros.
temperatura_c	DECIMAL(4,1)	NULL	Temperatura corporal en grados Celsius.
frecuencia_cardiaca	SMALLINT	NULL	Frecuencia cardíaca en latidos por minuto.
frecuencia_respiratoria	SMALLINT	NULL	Frecuencia respiratoria en respiraciones por minuto.
presion_sistolica	SMALLINT	NULL	Presión arterial sistólica en mmHg.
presion_diastolica	SMALLINT	NULL	Presión arterial diastólica en mmHg.
saturacion_oxigeno	DECIMAL(4,1)	NULL	Saturación de oxígeno en porcentaje (SpO2).
fecha_toma	TIMESTAMPTZ	DEFAULT NOW()	Momento exacto de la toma de signos vitales.

32. prescripciones

Propósito:

Generación de receta médica nominativa electrónica. MedIA genera la orden; el surtido físico es responsabilidad del sistema de farmacia externo vía campo puente codigo_medicament

o_ssa. Auditado por historial_cambios.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
id_prescripcion	UUID	PK	Folio interno de la receta.
id_encuentro	UUID	FK (encuentros_clinicos), NOT NULL	Encuentro generador de la prescripción.
id_medicamento	UUID	FK (cat_medicamentos), NOT NULL	Fármaco seleccionado del Cuadro Básico.
dosis	VARCHAR(50)	NOT NULL	Cantidad y unidad de dosis indicada. Ej: 500mg.
via_administracion	VARCHAR(50)	NOT NULL	Vía: ORAL, INTRAVENOSA, TOPICA, etc.
frecuencia	VARCHAR(100)	NOT NULL	Frecuencia de administración. Ej: Cada 8 horas.
duracion_dias	SMALLINT	NULL	Duración del tratamiento en días.
indicaciones	TEXT	NULL	Indicaciones adicionales para el paciente.
id_medico_prescriptor	UUID	FK (usuarios_sistema), NOT NULL	Médico que prescribe. Validado contra cédula_profesional.

33. solicitudes_estudio

Propósito: Órdenes de análisis clínicos (fase pre-analítica). MedIA genera la solicitud; el procesamiento de muestras es externo. Comunicación mediante campo puente tipo_estudio.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
id_solicitud	UUID	PK	Folio de la orden de laboratorio.
id_encuentro	UUID	FK (encuentros_clinicos), NOT NULL	Encuentro que genera la solicitud.
tipo_estudio	VARCHAR(100)	NOT NULL	Tipo de análisis solicitado. Ej: Biometría Hemática, Química Sanguínea.
indicaciones_clinicas	TEXT	NULL	Información clínica relevante para el laboratorio.
urgente	BOOLEAN	DEFAULT FALSE	Marca el estudio como prioritario.
id_medico_solicitante	UUID	FK (usuarios_sistema), NOT NULL	Médico que realiza la solicitud.
fecha_solicitud	TIMESTAMPTZ	DEFAULT NOW()	Fecha y hora de la solicitud.

34. resultados_laboratorio

Propósito:
Almacenamiento de valores reportados por sistemas de laboratorio externos. Referenciados mediante la solicitud original. Documentos PDF almacenados en Azure Blob Storage.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_resultado	UUID	PK	ID único del resultado.
id_solicitud	UUID	FK (solicitudes_estudio), NOT NULL	Referencia a la orden original. Garantiza trazabilidad de la cadena clínica.
valor_resultado	VARCHAR(255)	NOT NULL	Resultado numérico o textual reportado.
unidades	VARCHAR(50)	NULL	Unidades del resultado. Ej: mg/dL, g/L.
valores_referencia	VARCHAR(100)	NULL	Rango de referencia del laboratorio.
interpretacion	ENUM	'NORMAL','BAJO','ALTO','CRITICO', NULL	Interpretación automática del resultado.
documento_adjunto_url	TEXT	NULL	URL de Azure Blob Storage con el PDF completo del reporte de laboratorio.
fecha_resultado	TIMESTAMPTZ	NOT NULL	Fecha y hora en que el laboratorio reportó el resultado.

GRUPO 9 — Coordinación Asistencial (1 entidad)

35. referencias_medicas

Propósito:
Implementa el Sistema de Referencia y Contrarreferencia (SRC) normado por la Secretaría de Salud. Es el mecanismo legal que habilita el acceso cross-establecimiento o a notas clínicas. Sin una referencia activa, ningún médico de otro establecimiento puede acceder a las notas clínicas del

paciente, conforme a NOM-004 y la Ley General de Protección de Datos Personales.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
id_referencia	UUID	PK, DEFAULT gen_random_uuid()	ID único de la referencia.
id_encuentro_origen	UUID	FK (encuentros_clinicos), NOT NULL	Encuentro clínico que genera la necesidad de referencia.
id_establecimiento_destino	UUID	FK (establecimientos), NOT NULL	Unidad médica receptora del paciente.
id_especialidad_destino	SMALLINT	FK (cat_especialidades_medicas), NOT NULL	Especialidad a la que se refiere al paciente.
motivo_referencia	TEXT	NOT NULL	Resumen clínico: diagnóstico, tratamiento previo y razón de la referencia. Normado por NOM-004.
estado	ENUM	'EMITIDA','ACEPTADA','ATENDIDA','CANCELADA', NOT NULL	Estado del proceso de referencia.
fecha_emision	TIMESTAMPTZ	NOT NULL	Fecha de emisión de la referencia.
fecha_atencion	TIMESTAMPTZ	NULL	Fecha en que el establecimiento destino atendió al paciente.
id_medico_emisor	UUID	FK (usuarios_sistema), NOT NULL	Médico que emite la referencia.
resumen_contrarreferencia	TEXT	NULL	Informe del establecimiento receptor al emisor sobre el resultado de la atención.

GRUPO 10 — Seguridad Activa (4 entidades)

36. auditoria_accesos

Propósito: Bitácora inmutable de toda acción sobre datos de salud. Cumple NOM-024 numeral 4.1. El trigger tr_audit_no_changes prohíbe UPDATE y DELETE. Extendida con clasificación de incidentes de seguridad y trazabilidad geográfica cross-establecimiento.

Campo	Tipo PostgreSQL	Restricciones	Descripción
-------	-----------------	---------------	-------------

id_audit	BIGSERIAL	PK, NOT NULL	ID secuencial inalterable. BIGSERIAL garantiza orden cronológico.
id_usuario	UUID	FK (usuarios_sistema), NULL	Quién accedió. NULL en ataques externos sin autenticación.
accion	VARCHAR(100)	NOT NULL	Acción realizada: CREATE, READ, SIGN, UPDATE, DELETE_ATTEMPT.
tipo_evento	VARCHAR(50)	NOT NULL	Categoría del evento: LOGIN_FALLIDO, ACCESO_FUERA_HORARIO, INTENTO_XSS, INTENTO_SQLI, INTENTO_IDOR, ACCESO_SIN_REFERENCIA, MODIFICACION_NO_AUTORIZADA, CUENTA_BLOQUEADA, ACCESO_DENEGADO_ESPECIALIDAD.
nivel_severidad	ENUM	'BAJO','MEDIO','ALTO','CRITICO', NOT NULL	Nivel de riesgo del evento para priorización en el módulo de incidentes.
descripcion_evento	TEXT	NULL	Descripción detallada del evento para análisis posterior.
modulo_afectado	VARCHAR(100)	NULL	Módulo del sistema donde ocurrió el evento.
terminal_ip	INET	NOT NULL	Dirección IP del cliente. Tipo INET de PostgreSQL para validación nativa.
hash_archivo	CHAR(64)	NULL	SHA-256 hexadecimal del archivo involucrado en la acción. Aplica cuando la acción involucra un documento en Azure Blob Storage (resultado de laboratorio, documento de tutor). Permite verificar integridad del archivo en cualquier momento futuro. Req 1 Cómputo Forense.
modulo_funcion	VARCHAR(150)	NULL	Capa y función que ejecutó la acción. Ej: 'notas_medicas.service::firmar_nota'. Permite responder: ¿quién firmó?, ¿qué endpoint exportó?. Req 3 Cómputo Forense — Trazabilidad del código.
id_establecimiento_origen	UUID	FK (establecimientos), NULL	Establecimiento físico desde donde se originó el acceso.
id_establecimiento_dato	UUID	FK (establecimientos), NULL	Establecimiento propietario del dato que fue accedido.
justificacion_acceso	TEXT	NULL	Justificación para accesos cross-establecimiento. Normado por NOM-004.
fecha_hora	TIMESTAMPZ	DEFAULT NOW()	Marca de tiempo inalterable del evento.

37. incidentes_seguridad

Propósito: Gestión mutable del ciclo de vida de incidentes. SEPARADA de auditoria_accesos por diseño: la bitácora es inmutable (NOM-004), pero el estado del incidente debe poder actualizarse. Cumple los requisitos del módulo de erradicación de amenazas.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_incidente</code>	UUID	PK	ID único del incidente.
<code>id_auditoria_origen</code>	BIGINT	FK (auditoria_accesos), NOT NULL	Registro de auditoría que originó el incidente.
<code>estado</code>	ENUM	'ABIERTO','EN_PROCESO','RESUELTO','CERRADO', NOT NULL	Estado actual del ciclo de vida del incidente.
<code>vulnerabilidad_explotada</code>	VARCHAR(50)	NULL	Tipo de vulnerabilidad: XSS, IDOR, SQLI, BRUTE_FORCE, UNAUTHORIZED_ACCESS.
<code>descripcion_correccion</code>	TEXT	NULL	Descripción del procedimiento de erradicación aplicado.
<code>responsable_cierre</code>	UUID	FK (usuarios_sistema), NULL	Auditor que cierra el incidente.
<code>validacioneliminacion</code>	TEXT	NULL	Evidencia de que la amenaza fue eliminada. Requerido para cambiar estado a CERRADO.
<code>fecha_cierre_tecnico</code>	TIMESTAMPTZ	NULL	Fecha de cierre técnico del incidente.
<code>fecha_creacion</code>	TIMESTAMPTZ	DEFAULT NOW()	Fecha en que se abrió el incidente.

38. sesiones_invalidas

Propósito: JWT Blacklist. Registra tokens JWT revocados antes de su expiración natural. Necesario porque JWT es stateless por diseño: sin esta tabla, un token comprometido seguiría siendo válido hasta su expiración. Consultada por el

middleware de autenticación en cada petición.			
Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>jti</code>	UUID	PK	JWT ID único del token revocado. Campo <code>jti</code> del payload del JWT.
<code>id_usuario</code>	UUID	FK (usuarios_sistema), NOT NULL	Usuario al que pertenecía el token.
<code>invalidado_en</code>	TIMESTAMPTZ	DEFAULT NOW()	Momento en que se revocó el token.
<code>motivo</code>	VARCHAR(100)	NOT NULL	Razón de la revocación: SESION_SOSPECHOSA, RESET_CREDENCIALES, CIERRE_MANUAL, INCIDENTE_SEGURIDAD.
<code>expira_en</code>	TIMESTAMPTZ	NOT NULL	Expiración original del token. Permite limpiar registros obsoletos periódicamente.

39. bitacora_recuperacion

Propósito: Registro de eventos de recuperación de datos y servicios. Requerido por el documento de seguridad para demostrar el ciclo completo de recuperación controlada. Se activa en procedimientos de restauración PITR en Azure.

Campo	Tipo PostgreSQL	Restricciones	Descripción
<code>id_recuperacion</code>	UUID	PK	ID único del evento de recuperación.
<code>fecha_inicio</code>	TIMESTAMPTZ	NOT NULL	Inicio del procedimiento de recuperación.
<code>fecha_fin</code>	TIMESTAMPTZ	NULL	Fin del procedimiento. NULL indica recuperación en curso.
<code>tipo_incidente_origen</code>	VARCHAR(100)	NULL	Tipo de incidente que motivó la recuperación.
<code>descripcion_procedimiento</code>	TEXT	NOT NULL	Descripción detallada del procedimiento aplicado (PITR, failover, restauración de réplica).
<code>responsable</code>	UUID	FK (usuarios_sistema), NOT NULL	Administrador responsable del procedimiento.
<code>resultado</code>	ENUM	'EXITOSO', 'FALLIDO', 'PARCIAL', NOT NULL	Resultado de la recuperación.
<code>validacion_integridad</code>	TEXT	NULL	Resultados del script de verificación de integridad post-recuperación.

GRUPO 11 — Trazabilidad de Cambios (1 entidad)

40. historial_cambios

Propósito: Registro del estado anterior y nuevo de cualquier campo modificado en tablas clínicas y de seguridad. Implementado mediante triggers automáticos de PostgreSQL. Permite reconstrucción completa del expediente en cualquier punto del tiempo para defensa legal, cumpliendo NOM-004 y los requisitos del documento de seguridad de incidentes.

Campo	Tipo PostgreSQL	Restricciones	Descripción
id_cambio	UUID	PK	ID único del registro de cambio.
tabla_afectada	VARCHAR(100)	NOT NULL	Tabla donde ocurrió el cambio: personas, pacientes, alergias, antecedentes_*, signos_vitales, prescripciones, diagnosticos_encuentro, permisos_rol, establecimientos_especialidades, referencias_medicas, usuarios_sistema.
id_registro	UUID	NOT NULL	PK del registro que fue modificado.
campo_modificado	VARCHAR(100)	NOT NULL	Nombre de la columna que cambió.
valor_anterior	TEXT	NULL	Valor previo serializado como texto. NULL en INSERT.
valor_nuevo	TEXT	NOT NULL	Valor nuevo serializado como texto.
id_usuario	UUID	FK (usuarios_sistema), NOT NULL	Usuario que realizó el cambio.
id_establecimiento	UUID	FK (establecimientos), NOT NULL	Establecimiento desde donde se realizó el cambio.
fecha_cambio	TIMESTAMPTZ	DEFAULT NOW()	Timestamp exacto del cambio. Generado por el trigger, no por el cliente.
motivo_cambio	TEXT	NULL	Justificación del cambio si fue ingresada por el usuario.

Sección 2 — Reglas de Inalterabilidad y Triggers

Para cumplir con la NOM-004-SSA3-2012, los registros clínicos firmados no pueden ser editados ni eliminados. Se implementan cinco triggers en PostgreSQL que garantizan estas reglas de forma estructural, independientemente de la capa de aplicación.

Trigger 1 — Inmutabilidad de la Bitácora de Auditoría

Bloquea cualquier intento de UPDATE o DELETE en la tabla auditoria_accesos. La bitácora es un registro médico-legal que no puede ser alterado bajo ninguna circunstancia.

```
CREATE OR REPLACE FUNCTION fn_audit_immutability()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'La tabla de auditoria es inmutable por NOM-024-SSA3-2012.
Operacion rechazada.';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_audit_no_changes
BEFORE UPDATE OR DELETE ON auditoria_accesos
FOR EACH ROW EXECUTE FUNCTION fn_audit_immutability();
```

Trigger 2 — Bloqueo de Notas Clínicas Firmadas

Impide la modificación de una nota clínica una vez que el campo esta_firmada es TRUE. Implementa el principio de inmutabilidad del acto médico documentado conforme a NOM-004 numeral 5.10.

```
CREATE OR REPLACE FUNCTION fn_lock_signed_notes()
RETURNS TRIGGER AS $$
BEGIN
    IF (OLD.esta_firmada = TRUE) THEN
        RAISE EXCEPTION 'Registro bloqueado. La nota ya cuenta con firma electronica
(NOM-004).';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_notes_protection
BEFORE UPDATE ON notas_medicas
FOR EACH ROW EXECUTE FUNCTION fn_lock_signed_notes();
```

Trigger 3 — Historial de Cambios en Datos Clínicos

Función genérica reutilizable que registra automáticamente el estado anterior y nuevo de cualquier campo modificado en las tablas clínicas y de seguridad configuradas. Se aplica mediante triggers individuales en cada tabla auditada.

```
CREATE OR REPLACE FUNCTION fn_registrar_cambio()
RETURNS TRIGGER AS $$
DECLARE
    col_name TEXT;
    val_antes TEXT;
```

```

    val_despues TEXT;
BEGIN
    FOR col_name IN SELECT column_name FROM information_schema.columns
        WHERE table_name = TG_TABLE_NAME
    LOOP
        EXECUTE format('SELECT ($1).%I::TEXT', col_name) INTO val_antes USING OLD;
        EXECUTE format('SELECT ($1).%I::TEXT', col_name) INTO val_despues USING NEW;
        IF val_antes IS DISTINCT FROM val_despues THEN
            INSERT INTO historial_cambios (
                tabla_afectada, id_registro, campo_modificado,
                valor_anterior, valor_nuevo, id_usuario, id_establecimiento
            ) VALUES (
                TG_TABLE_NAME, NEW.id_paciente, col_name,
                val_antes, val_despues, current_setting('app.current_user_id')::UUID,
                current_setting('app.current_establecimiento_id')::UUID
            );
        END IF;
    END LOOP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Aplicar a cada tabla auditada:
CREATE TRIGGER tr_historial_alergias
    AFTER UPDATE ON alergias FOR EACH ROW EXECUTE FUNCTION fn_registrar_cambio();

-- Replicar para: personas, pacientes, antecedentes_heredofamiliares,
-- antecedentes_patologicos, antecedentes_no_patologicos,
-- antecedentes_ginecoobstetricos, signos_vitales, prescripciones,
-- diagnosticos_encuentro, usuarios_sistema, referencias_medicas,
-- establecimientos_especialidades, permisos_rol

```

Trigger 4 — Bloqueo de Contención por Intentos Fallidos

Activa el bloqueo temporal de una cuenta cuando el contador de intentos_fallidos alcanza el umbral definido (5 intentos). Implementa el mecanismo de contención automática requerido por el documento de seguridad.

```

CREATE OR REPLACE FUNCTION fn_bloqueo_cuenta()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.intentos_fallidos >= 5 THEN
        NEW.bloqueado_hasta := NOW() + INTERVAL '30 minutes';
        INSERT INTO auditoria_accesos (id_usuario, accion, tipo_evento,
            nivel_severidad,
            descripcion_evento, terminal_ip)
        VALUES (NEW.id_usuario, 'BLOCK', 'CUENTA_BLOQUEADA', 'ALTO',
            'Cuenta bloqueada automaticamente tras 5 intentos fallidos.',
            inet_client_addr());
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_bloqueo_por_intentos
    BEFORE UPDATE OF intentos_fallidos ON usuarios_sistema
    FOR EACH ROW EXECUTE FUNCTION fn_bloqueo_cuenta();

```

Trigger 5 — Alerta de Incidente Crítico

Crea automáticamente un registro en incidentes_seguridad cuando se registra un evento con nivel_severidad CRITICO o ALTO en la bitácora de auditoría, iniciando el ciclo de vida del incidente para atención del Auditor de Seguridad.

```
CREATE OR REPLACE FUNCTION fn_crear_incidente()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.nivel_severidad IN ('ALTO', 'CRITICO') THEN
        INSERT INTO incidentes_seguridad (id_auditoria_origen, estado,
        vulnerabilidad_explotada)
        VALUES (NEW.id_audit, 'ABIERTO', NEW.tipo_evento);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_incidente_automatico
AFTER INSERT ON auditoria_accesos
FOR EACH ROW EXECUTE FUNCTION fn_crear_incidente();
```

Trigger 6 — Registro Forense de Borrado Lógico

Cuando se ejecuta un borrado lógico (soft delete) en cualquier tabla clínica, este trigger registra automáticamente el evento en auditoria_accesos con nivel_severidad ALTO, cumpliendo el Req 6 de Cómputo Forense. El borrado físico (DELETE) está prohibido en tablas clínicas mediante permisos de base de datos — el único mecanismo de 'eliminación' permitido es actualizar eliminado_en.

```
CREATE OR REPLACE FUNCTION fn_log_soft_delete()
RETURNS TRIGGER AS $$
BEGIN
    -- Solo dispara cuando eliminado_en cambia de NULL a un valor
    IF OLD.eliminado_en IS NULL AND NEW.eliminado_en IS NOT NULL THEN
        INSERT INTO auditoria_accesos (
            id_usuario, accion, tipo_evento, nivel_severidad,
            descripcion_evento, terminal_ip, modulo_funcion
        ) VALUES (
            NEW.eliminado_por,
            'SOFT_DELETE',
            'BORRADO_LOGICO',
            'ALTO',
            format('Borrado lógico en %s. ID: %s. Motivo: %s',
                TG_TABLE_NAME, OLD.id_paciente, NEW.motivo_baja),
            inet_client_addr(),
            TG_TABLE_NAME || '::soft_delete'
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Aplicar a todas las tablas con soft delete:
CREATE TRIGGER tr_soft_delete_alergias
AFTER UPDATE OF eliminado_en ON alergias
FOR EACH ROW EXECUTE FUNCTION fn_log_soft_delete();

-- Replicar para: inmunizaciones, antecedentes_heredofamiliares,
```

```
-- antecedentes_patologicos, antecedentes_no_patologicos,  
-- antecedentes_ginecoobstetricos, pacientes_tutores_representantes
```

Sección 3 — Gestión de Catálogos y Normalización

Los catálogos de MedIA se implementan como entidades estáticas sembradas en la fase de inicialización del sistema (seed). Se clasifican en tres tipos:

Tipo	Catálogos	Comportamiento
Estáticos puros	cat_cie10, cat_medicamentos	Nunca se modifican en operación. ON DELETE RESTRICT protege referencias históricas.
Geográficos INEGI	cat_estados, cat_municipios, cat_localidades	Sembrados con catálogo INEGI. Se actualizan por carga masiva cuando INEGI publica nueva versión.
Configuración	establecimientos_especialidades, permisos_rol	Modificables por Administrador desde el panel. Cambios auditados en historial_cambios.

Integridad referencial: Todos los catálogos clínicos aplican ON DELETE RESTRICT en sus FK. Esto garantiza que un diagnóstico CIE-10 o medicamento no pueda eliminarse si ya forma parte de un encuentro clínico previo, preservando la integridad histórica del expediente.

Sección 4 — Persistencia de Archivos y Generación de Documentos

MedIA implementa una política clara sobre el origen de los documentos clínicos: los documentos del expediente son GENERADOS por el sistema a partir de datos ya registrados, nunca subidos manualmente por el médico. Esta decisión garantiza autoría verificable, timestamp confiable del servidor y firma electrónica del sistema — cumpliendo NOM-004 y los principios forenses del Req 1.

Documentos Generados por el Sistema (PDF desde datos registrados)

Documento	Estado MVP	Origen de datos	Justificación
Receta médica nominativa	✓ Includido	<code>prescripciones + usuarios_sistema</code>	NOM-004 requiere folio único, nombre del médico y cédula. Generado desde datos ya validados.
Nota de consulta SOAP	✓ Includido	<code>notas_soap_detalle + notas_medicas</code>	PDF generado al momento de firmar. Inmutable desde ese instante.
Solicitud de laboratorio	✓ Includido	<code>solicitudes_estudio + encuentros_clinicos</code>	Orden formal con folio trazable al encuentro clínico.
Resumen de referencia médica	✓ Includido	<code>referencias_medicas + pacientes</code>	Documento normado por el SRC. Incluye diagnóstico y médico emisor.
Constancia médica / Incapacidad	⌚ Trabajo futuro	—	Documento clínico válido pero agrega complejidad de plantillas fuera del alcance de 9 semanas.

Archivos Externos — Azure Blob Storage (único caso de upload)

El único archivo que entra al sistema desde una fuente externa es el resultado de laboratorio en PDF, proveniente del sistema externo de laboratorio. Este es el único caso donde aplica el campo `hash_archivo` en `auditoria_accesos`. El médico nunca sube documentos clínicos — esa responsabilidad queda en el sistema generador.

Entidad	Campo de Referencia	Propósito
<code>pacientes_tutores_representantes</code>	<code>documento_legal_url</code>	Escaneo del documento que acredita la representación legal. Hash SHA-256 registrado en <code>auditoria_accesos</code> al momento del upload.
<code>resultados_laboratorio</code>	<code>documento_adjunto_url</code>	PDF del reporte de laboratorio externo. Es el único caso de documento clínico de origen externo. Hash SHA-256 obligatorio en <code>auditoria_accesos</code> .

Sección 5 — Seguridad y Control de Acceso (RBAC)

El control de acceso basado en roles (RBAC) se implementa mediante tres capas complementarias. Esta arquitectura multicapa cumple NOM-024-SSA3-2012 numeral 4.1 y los requisitos de seguridad del documento de incidentes.

Capa 1 — Permisos por Módulo (permisos_rol)

Define qué operaciones CRUD puede realizar cada rol sobre cada módulo del sistema. Administrable desde la interfaz sin intervención técnica.

Rol	Módulo	Crear	Leer	Actualizar	Eliminar	Nota
MEDICO_GENERAL	NOTAS_MEDICAS	✓	✓	✗	✗	Inmutable por NOM-004
MEDICO_GENERAL	PACIENTES	✓	✓	✓	✗	Auditado en historial
MEDICO_ESPECIALISTA	NOTAS_MEDICAS	✓	✓	✗	✗	Solo su especialidad
ENFERMERIA	SIGNOS_VITALES	✓	✓	✓	✗	Antes del cierre
ENFERMERIA	NOTAS_MEDICAS	✗	✓	✗	✗	Solo lectura
ADMINISTRADOR	CONFIGURACION	✓	✓	✓	✗	Sin acceso a clínica
ADMINISTRADOR	NOTAS_MEDICAS	✗	✗	✗	✗	Sin acceso clínico
AUDITOR_SEGURIDAD	AUDITORIA	✗	✓	✗	✗	Solo lectura bitácara

Capa 2 — Visibilidad por Especialidad (permisos_especialidad)

Restringe qué notas puede ver un médico en accesos cross-establecimiento según la especialidad generadora. Dentro del mismo establecimiento, el expediente es único y compartido conforme a NOM-004.

Capa 3 — Acceso Cross-Establecimiento (referencias_medicas)

Las notas clínicas de otro establecimiento solo son accesibles si existe una referencia médica con estado ACEPTADA o ATENDIDA. Los datos de identidad y seguridad (personas, alergias, antecedentes) son accesibles para cualquier médico con un encuentro activo abierto para el paciente, independientemente del establecimiento de origen.

VIEWS de Seguridad

Se crean vistas por nivel de acceso para limitar la exposición de datos sensibles conforme al documento de seguridad:

```
-- Vista para roles administrativos: sin CURP ni datos clínicos
CREATE VIEW v_paciente_basico AS
SELECT id_paciente, numero_expediente,
       p.nombre, p.primer_apellido, p.fecha_nacimiento
FROM pacientes pac
```

```
JOIN personas p ON pac.id_persona = p.id_persona;

-- Vista para auditorías estadísticas: sin identificación de usuarios
CREATE VIEW v_auditoria_estadistica AS
  SELECT tipo_evento, nivel_severidad, modulo_afectado,
    DATE(fecha_hora) as fecha, COUNT(*) as total
  FROM auditoria_accesos
  GROUP BY tipo_evento, nivel_severidad, modulo_afectado, DATE(fecha_hora);

-- Vista para enfermería: signos vitales sin notas SOAP
CREATE VIEW v_signos_encuentro AS
  SELECT sv.*, e.id_paciente, e.fecha_inicio
  FROM signos_vitales sv
  JOIN encuentros_clinicos e ON sv.id_encuentro = e.id_encuentro;
```

Sección 7 — Diagrama de Relaciones (Topología del Modelo)

El modelo se organiza en cuatro dominios funcionales con relaciones claramente definidas:

Dominio 1 — Identidad Base y Geografía

```
cat_estados (1) → (N) cat_municipios
cat_municipios (1) → (N) cat_localidades
cat_localidades (1) → (N) personas
cat_lenguas_indigenas (1) → (N) personas
personas (1) → (1) pacientes
personas (1) → (1) usuarios_sistema
personas (1) → (N) pacientes_tutores_representantes
```

Dominio 2 — Red Asistencial y Control de Acceso

```
jurisdicciones_sanitarias (1) → (N) establecimientos
establecimientos (1) → (N) establecimientos_especialidades
cat_especialidades_medicas (1) → (N) establecimientos_especialidades
roles (1) → (N) permisos_rol
cat_modulos (1) → (N) permisos_rol
roles (1) → (N) permisos_especialidad
cat_especialidades_medicas (1) → (N) permisos_especialidad
usuarios_sistema (1) → (N) usuarios_establecimientos
establecimientos (1) → (N) usuarios_establecimientos
roles (1) → (N) usuarios_establecimientos
```

Dominio 3 — Historia Clínica del Paciente

```
pacientes (1) → (N) alergias
pacientes (1) → (N) antecedentes_heredofamiliares
pacientes (1) → (N) antecedentes_patologicos
pacientes (1) → (N) antecedentes_no_patologicos
pacientes (1) → (1) antecedentes_ginecoobstetricos
pacientes (1) → (N) inmunizaciones
pacientes (1) → (N) pacientes_tutores_representantes
```

Dominio 4 — Núcleo del Acto Médico

```
pacientes (1) → (N) encuentros_clinicos
establecimientos (1) → (N) encuentros_clinicos
encuentros_clinicos (1) → (N) notas_medicas
notas_medicas (1) → (1) notas_soap_detalle
notas_medicas (1) → (N) notas_enmienda
encuentros_clinicos (1) → (N) diagnosticos_encuentro
cat_cie10 (1) → (N) diagnosticos_encuentro
encuentros_clinicos (1) → (N) signos_vitales
encuentros_clinicos (1) → (N) prescripciones
cat_medicamentos (1) → (N) prescripciones
encuentros_clinicos (1) → (N) solicitudes_estudio
solicitudes_estudio (1) → (N) resultados_laboratorio
```

encuentros_clinicos (1) → (N) referencias_medicas

Sección 6 — Cumplimiento de Requerimientos de Cómputo Forense

El modelo de base de datos cubre de forma estructural los 9 requerimientos del documento de Cómputo Forense de la BUAP. Los requerimientos que no requieren cambio en el modelo de BD se resuelven en la capa de aplicación (FastAPI) y se documentan aquí para trazabilidad del diseño.

#	Requerimiento	Estado	Implementación en MedIA
1	Registro de eventos (Logging forense)	✓ BD + App	auditoria_accesos: usuario, timestamp TIMESTAMPTZ, acción, resultado, IP (INET), hash_archivo SHA-256, modulo_funcion. Inmutable por trigger tr_audit_no_changes.
2	Manejo correcto de excepciones	✓ App	FastAPI captura excepciones de triggers PostgreSQL y las registra en auditoria_accesos antes de devolver HTTP 400/403. El error queda documentado sin alterar evidencia.
3	Trazabilidad del código	✓ BD + App	Campo modulo_funcion en auditoria_accesos. Permite responder: '¿qué endpoint firmó la nota?', '¿qué función generó la prescripción?'. Capas separadas: UI/API/Service/Repository.
4	Marcas de tiempo confiables	✓ BD	Todos los timestamps usan TIMESTAMPTZ (UTC). Generados por triggers del servidor PostgreSQL, no por el cliente. Azure sincroniza con NTP. Formato ISO 8601 nativo.
5	Identificación de acciones críticas	✓ BD + Doc	nivel_severidad ENUM en auditoria_accesos. Lista documentada de acciones críticas: firmar nota, crear usuario, borrado lógico, cambio de permisos, reset de credenciales.
6	No borrar ni sobrescribir evidencia	✓ BD	Soft delete en 7 tablas clínicas: eliminado_en, eliminado_por, motivo_baja. Trigger tr_soft_delete_* registra cada borrado lógico. DELETE físico prohibido en tablas clínicas por permisos de BD.
7	Separación de responsabilidades	✓ BD + Arq	Roles: MEDICO_GENERAL, MEDICO_ESPECIALISTA, ENFERMERIA, ADMINISTRADOR, AUDITOR_SEGURIDAD. Separación UI/FastAPI/PostgreSQL. AUDITOR_SEGURIDAD solo puede leer auditoria, no datos clínicos.
8	Uso responsable de configuraciones	✓ Infra	Credenciales en variables de entorno Azure App Service. password_hash con Argon2id. totp_secret cifrado. Sin hardcoding. SAS tokens de 15 min para Blob Storage.
9	Comentarios técnicos relevantes	✓ Código	Cada función crítica del backend documenta: justificación técnica, control implementado y referencia normativa (NOM-004, NOM-024, Req Forense). Aplicado en la fase de codificación.

Sección 8 — Distribución Centralizada vs. Distribuida (Cómputo Distribuido)

El modelo de distribución de datos responde a los principios de cómputo distribuido aplicados al contexto geográfico del Distrito de Salud I: unidades médicas urbanas con buena conectividad y clínicas rurales con conectividad limitada o intermitente.

Entidad / Grupo	Clasificación	Justificación y Estrategia
notas_medicas, pacientes, auditoria_accesos	Centralizada	Datos maestros clínicos con consistencia fuerte requerida. Un paciente no puede existir en dos versiones simultáneas. Escritura siempre al servidor primario Azure.
cat_ciel0, cat_medicamentos, cat_lenguas_indigenas	Replicada (solo lectura)	Catálogos estáticos de consulta frecuente. Se cachean en nodos CDN regionales. Invalidación manual cuando la DGIS publica actualización del CIE-10.
cat_estados, cat_municipios, cat_localidades	Replicada (solo lectura)	Catálogos geográficos INEGI. Consulta frecuente en formularios de registro. Sin cambios operacionales.
encuentros_clinicos, signos_vitales	Centralizada con acceso distribuido	Escritura siempre al servidor central. Lectura distribuida vía Azure Front Door que enruta al nodo con menor latencia para el establecimiento solicitante.
jurisdicciones_sanitari as, establecimientos	Replicada (solo lectura)	Datos administrativos de referencia necesarios en todos los nodos para validar accesos y rutas de referencia.
auditoria_accesos, historial_cambios	Centralizada estricta	Jamás se replican ni distribuyen. Datos de auditoría médico-legal deben residir en un único nodo con control total.

Sincronización y Descomposición de Consultas

Sincronización: Las escrituras clínicas se dirigen siempre al servidor primario en Azure (East US). Los catálogos replicados en CDN se invalidan mediante una llamada al endpoint de administración cuando el coordinador del Distrito actualiza un catálogo.

Descomposición de consultas: Una consulta de 'historial completo de paciente' se descompone internamente en: (1) datos demográficos desde caché CDN si disponible, (2) antecedentes desde servidor primario, (3) notas SOAP desde servidor primario con validación de referencia médica activa. Los pasos 2 y 3 nunca se cachean por ser datos clínicos sensibles.

Sección 9 — Reglas de Negocio Fundamentales

Las siguientes cuatro reglas gobiernan el comportamiento del sistema en la capa de aplicación (backend FastAPI). Son las reglas que convierten el modelo de datos en un sistema normativamente sólido y no solo en un esquema de tablas relacionadas.

Regla 1 — Datos de Identidad y Seguridad son Universales

Las tablas personas, alergias y todos los antecedentes son visibles para cualquier médico autenticado que tenga un encuentro_clinico activo (fecha_cierre = NULL) abierto para ese paciente, independientemente del establecimiento de origen de los datos. Justificación normativa: la NOM-004 considera estos datos como información de seguridad clínica indispensable para cualquier acto médico. Un médico no puede atender de forma segura a un paciente sin conocer sus alergias.

Regla 2 — Notas Clínicas son Propiedad del Establecimiento

Las notas_medicas y notas_soap_detalle de un establecimiento A solo son accesibles para un médico del establecimiento B si existe en referencias_medicas un registro con estado ACEPTADA o ATENDIDA que vincule al paciente con el establecimiento B. Sin referencia activa, el acceso devuelve HTTP 403 y genera un registro en auditoria_accesos con tipo_evento = 'ACCESO_SIN_REFERENCIA'. Fundamento: NOM-004 + Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados.

Regla 3 — La Especialidad del Establecimiento Limita el Acceso Cross-Establecimiento

Incluso con referencia médica activa, el médico del establecimiento destino solo accede a las notas de las especialidades que su propio establecimiento tiene registradas como activas en establecimientos_especialidades. Un odontólogo con referencia puede ver notas de odontología y antecedentes, pero no notas de ginecología. Esta regla refleja la realidad clínica: un establecimiento sin capacidad instalada en ginecología no tiene necesidad asistencial de ver esos datos.

Regla 4 — Todo Acceso Queda Registrado Sin Excepción

Cada operación de lectura, escritura, intento fallido y denegación genera un registro en auditoria_accesos con el contexto completo: id_establecimiento_origen (desde dónde), id_establecimiento_dato (de quién es el dato), tipo_evento y nivel_severidad. Esta regla se implementa como middleware en FastAPI antes de que la petición llegue a la lógica de negocio, garantizando que incluso los intentos bloqueados queden registrados.

Sección 10 — Contexto Chiapas y Decisiones de Diseño

Identidad sin CURP — Poblaciones Vulnerables

Dada la realidad demográfica del estado, la CURP en la tabla personas es NULL permitido. El sistema genera automáticamente un numero_expediente basado en la fecha de atención y un hash UUID aleatorio para pacientes que no cuenten con CURP al momento de la consulta. Esto garantiza que ningún paciente sea excluido del sistema por carecer de documentación.

Domicilio Estructurado con Catálogo INEGI

El campo de domicilio no es TEXT libre. Se implementa mediante FK a cat_localidades (catálogo INEGI) complementado con campos de calle y referencia_geografica. El campo referencia_geografica (NULL permitido) captura direcciones no formales: 'Frente a la cancha ejidal', 'A 2 km de la cabecera municipal'. Esto permite reportes epidemiológicos reales por municipio y localidad para la coordinación del Distrito.

Enfoque Intercultural de Primer Contacto

El registro de id_lengua_materna en personas activa una alerta visual en el módulo de encuentros_clinicos para que el personal médico identifique inmediatamente si existe una barrera lingüística total o parcial. Chiapas cuenta con más de 12 grupos etnolingüísticos activos según el INALI, y la comunicación médico-paciente es un componente crítico de la seguridad clínica.

Conectividad Rural y Timestamp de Creación

Las notas_medicas conservan el campo fecha_creacion con el timestamp real del momento en que el médico elaboró la nota, independientemente de cuándo se sincronice con el servidor central. Esto preserva el orden cronológico exigido por NOM-004 incluso en escenarios de reconexión tras pérdida de señal en clínicas rurales atendidas por Azure CDN y Azure Front Door.

Sección 11 — Criterios de Disponibilidad, Auditoría y Recuperación

Auditoría Inmutable

La tabla auditoria_accesos registra el contexto geográfico completo de cada acceso: IP del cliente, establecimiento de origen, establecimiento propietario del dato, tipo de evento y nivel de severidad. El trigger tr_audit_no_changes prohíbe cualquier operación de UPDATE o DELETE, haciendo la bitácora técnicamente inalterable a nivel de base de datos.

Firma Electrónica Simple

El bloqueo de notas médicas se activa mediante el flag esta_firmada. Una vez activo, el trigger tr_notes_protection impide cualquier alteración al cuerpo de la nota SOAP. Las correcciones posteriores se realizan exclusivamente mediante notas_enmienda, preservando el registro original visible conforme a NOM-004.

Resiliencia y Recuperación — Azure PostgreSQL

Azure Database for PostgreSQL proporciona respaldos automáticos con retención configurable (7–35 días) y Point-In-Time Recovery (PITR) con granularidad de 5 minutos. El procedimiento de recuperación se documenta en bitacora_recuperacion. El plan de continuidad operativa contempla failover a réplica de lectura en caso de caída del servidor primario, con redirección del DATABASE_URL en Azure App Service.

Replicación de Base de Datos

La replicación no se implementa como código propio del equipo. Azure Database for PostgreSQL proporciona replicación síncrona y failover automático como característica nativa del servicio gestionado. Implementar replicación manualmente sobre esta plataforma duplicaría capacidad ya provista, introduciendo riesgos de inconsistencia innecesarios. Se documenta el procedimiento de configuración en Azure Portal como parte del manual de operaciones.

Resumen — Inventario de Entidades (40 tablas)

#	Grupo	Entidades	Total
1	Identidad Base	personas	1
2	Catálogos Geográficos INEGI	cat_estados, cat_municipios, cat_localidades	3
3	Catálogos Clínicos y de Sistema	jurisdicciones_sanitarias, establecimientos, cat_ciel0, cat_medicamentos, cat_especialidades_medicas, cat_lenguas_indigenas, cat_modulos	7
4	Configuración Red Asistencial	establecimientos_especialidades	1
5	Usuarios y Control de Acceso	usuarios_sistema, roles, permisos_rol, usuarios_establecimientos, permisos_especialidad	5
6	Identidad Clínica del Paciente	pacientes, pacientes_tutores_representantes, alergias	3
7	Historia Clínica — Antecedentes	antecedentes_heredofamiliares, antecedentes_patologicos, antecedentes_no_patologicos, antecedentes_ginecoobstetricos, inmunizaciones	5
8	Núcleo del Acto Médico	encuentros_clinicos, notas_medicas, notas_soap_detalle, notas_enmienda, diagnosticos_encuentro, signos_vitales, prescripciones, solicitudes_estudio, resultados_laboratorio	9
9	Coordinación Asistencial (SRC)	referencias_medicas	1
10	Seguridad Activa	auditoria_accesos, incidentes_seguridad, sesiones_invalidas, bitacora_recuperacion	4
11	Trazabilidad de Cambios	historial_cambios	1
TOTAL DE ENTIDADES:			40